

Research Plan: Formal Methods in Mathematics

Student Name: William Fayers

Project Supervisor: Dr. Yuri Santos Rego

Email: YSantosRego@lincoln.ac.uk

Module: MTH3011

Date: 27 October 2025

1. Project Title

Formal Methods in Mathematics: A Study of Computer-Verified Proofs Using Interactive Theorem Provers

2. Project Description and Introduction

Mathematics derives its foundational strength from rigorous logical frameworks built upon core axioms. However, as a fundamentally human endeavour, mathematical reasoning remains susceptible to errors even at the highest levels of research. Notable examples include flaws discovered in published proofs and theorems of exceptional complexity—such as the Feit-Thompson theorem—whose correctness is difficult for humans to verify comprehensively ^{[1] [2]}.

The emergence of proof assistants in the 1980s represented a transformative development in mathematical practice. These computer programmes, grounded in higher-order logic and dependent type theory, enable the formalisation of mathematical theories without ambiguity and provide rigorous verification of proof correctness ^{[3] [4]}. Modern systems such as Lean, Coq (Rocq), and the Prototype Verification System (PVS) have matured significantly, facilitating applications ranging from pure mathematics to the verification of safety-critical systems ^{[5] [6]}.

This project investigates formal methods in mathematics with particular emphasis on interactive theorem proving. The research will explore the logical foundations underpinning proof assistants, examine automated reasoning capabilities, and undertake practical implementations using contemporary systems. The scope encompasses both theoretical dimensions—including Gödel's incompleteness theorems and their implications for formal systems—and applied aspects such as computer-assisted proof development ^{[7] [8]}.

A motivating example illustrating the necessity of formal verification comes from cryptographic protocol analysis. The Needham-Schroeder protocol, a foundational authentication mechanism, contained a subtle flaw that remained undetected for nearly two decades until Gavin Lowe discovered it in 1995 ^{[9] [10]}. This vulnerability could be exploited by adversaries to impersonate legitimate parties. Formal verification techniques using systems like Isabelle/HOL have since been employed to analyse and verify corrected versions of such protocols, demonstrating the practical value of computer-verified proofs in ensuring system security ^{[11] [12]}.

3. Connection to Previous Studies

This project builds directly upon several modules completed during my undergraduate programme:

MTH2004 - Mathematical Structures: This module provided foundations in abstract algebra, group theory, and formal mathematical reasoning. Understanding algebraic structures is essential when working with proof assistants, as many formalised mathematical libraries begin with group theory and ring theory ^[13].

MTH2002 - Real Analysis: The rigorous treatment of limits, continuity, and convergence in this module established familiarity with epsilon-delta proofs and the level of detail required in formal mathematics. Interactive theorem provers demand similarly precise specifications ^[14].

MTH1003 - Foundations of Pure Mathematics: Introduction to logic, set theory, and proof techniques (induction, contradiction, contrapositive) provided essential preparation for understanding the logical frameworks underlying proof assistants ^[15].

Programming Experience: My substantial background in Python and TypeScript, combined with experience in API development and systems design, positions me well to engage with the computational aspects of theorem proving. Proof assistants utilise programming-language-like syntax, and many systems support code extraction for verified algorithm implementations ^{[16] [17]}.

Third-Year Modules: Current studies in numerical methods (MTH3006) provide relevant context for understanding how formal verification can ensure the correctness of computational algorithms, complementing traditional numerical analysis approaches ^[18].

4. Literature Survey

The following sources constitute the initial foundation for this research project:

Foundational Texts

Ayala-Rincon, M. & de Moura, F. L. C. (2017). *Applied Logic for Computer Scientists: Computational Deduction and Formal Proofs*. Springer ^[19].

This comprehensive textbook covers formal logic systems, deduction mechanisms, and proof theory essential for understanding automated reasoning.

Mendelson, E. (2015). *Introduction to Mathematical Logic* (6th ed.). CRC Press ^[20].

Provides thorough coverage of first-order logic, completeness, soundness, and foundational results including Gödel's incompleteness theorems.

Proof Assistant Systems

Geuvers, H. (2009). "Proof assistants: history, ideas and future." *Sadhana*, 34, 3-25 ^[21].

This survey article traces the historical development of proof assistants from early systems to modern implementations, discussing design philosophies and future directions.

de Moura, L., Kong, S., Avigad, J., van Doorn, F. & von Raumer, J. (2015). "The Lean Theorem Prover (system description)." *International Conference on Automated Deduction*, 378-388 ^[22].

Describes Lean's architecture, including its elaborator for higher-order unification and integration of automated reasoning tools within an interactive framework.

Formalisation Projects

Gonthier, G., et al. (2013). "A Machine-Checked Proof of the Odd Order Theorem." *Interactive Theorem Proving*, 163-179 ^[23].

Documents the six-year collaborative formalisation of the Feit-Thompson theorem in Coq, demonstrating that large-scale mathematical proofs are achievable with proof assistants.

Paulson, L. C. (2014). "A machine-assisted proof of Gödel's incompleteness theorems for the theory of hereditarily finite sets." *Review of Symbolic Logic*, 7(3), 484-498 ^[24].

Presents a constructive formalisation of Gödel's incompleteness results, illustrating how foundational meta-mathematical theorems can be verified mechanically.

Automated Reasoning

Harrison, J., Urban, J. & Wiedijk, F. (2014). "History of Interactive Theorem Proving." *Handbook of the History of Logic*, 9, 135-214 ^[25].

Comprehensive historical overview covering the development of automated and interactive theorem proving from the 1950s to contemporary systems.

Formal Verification Applications

Noce, P. (2020). "Formal Verification of Cryptographic Protocols with Isabelle/HOL." In *Formal Verification of Critical Systems*, 65-89. CRC Press ^[26].

Explores application of formal methods to protocol verification, including analysis of the Needham-Schroeder protocol and its variants.

Muñoz, C. & Narkawicz, A. (2013). "Formalization of Bernstein polynomials and applications to global optimization." *Journal of Automated Reasoning*, 51(2), 151-196 ^[27].

Demonstrates application of PVS to formalise numerical algorithms with rigorous correctness guarantees, relevant for understanding formal methods in computational mathematics.

Recent Developments

Buzzard, K., Commelin, J. & Massot, P. (2020). "The Lean mathematical library." *Proceedings of CPP 2020*, 367-381 ^[28].

Describes mathlib's architecture, design principles, and community organisation, providing insight into modern mathematical library development.

Hales, T. C. (2014). "Developments in Formal Proofs." *Séminaire Bourbaki*, 1086 ^[29].

Discusses technological advances in formal proof including reliable system design, large-scale formalisations, and automation improvements.

This literature survey will be expanded throughout the project as deeper investigation reveals additional relevant sources. Initial reading will focus on the foundational texts and proof assistant documentation, progressing to specific formalisation projects aligned with the chosen research focus.

5. References

^[1] Geuvers, H. (2009). Proof assistants: history, ideas and future. *Sadhana*, 34, 3-25.

^[2] Gonthier, G., et al. (2013). A Machine-Checked Proof of the Odd Order Theorem. *Interactive Theorem Proving*, 163-179.

^[3] de Moura, L., Kong, S., Avigad, J., van Doorn, F. & von Raumer, J. (2015). The Lean Theorem Prover (system description). *International Conference on Automated Deduction*, 378-388.

^[4] Owre, S., Rushby, J. M. & Shankar, N. (1992). PVS: A prototype verification system. *International Conference on Automated Deduction*, 748-752.

^[5] Buzzard, K., Commelin, J. & Massot, P. (2020). The Lean mathematical library. *Proceedings of CPP 2020*, 367-381.

^[6] NASA Langley Research Center. (2024). Langley Formal Methods Program. Retrieved from <https://shemesh.larc.nasa.gov/fm/pvs/>

^[7] Paulson, L. C. (2014). A machine-assisted proof of Gödel's incompleteness theorems for the theory of hereditarily finite sets. *Review of Symbolic Logic*, 7(3), 484-498.

^[8] Mendelson, E. (2015). *Introduction to Mathematical Logic* (6th ed.). CRC Press.

^[9] Lowe, G. (1996). Breaking and fixing the Needham-Schroeder public-key protocol using FDR. *Tools and Algorithms for the Construction and Analysis of Systems*, 147-166.

^[10] Needham, R. M. & Schroeder, M. D. (1978). Using encryption for authentication in large networks of computers. *Communications of the ACM*, 21(12), 993-999.

^[11] Noce, P. (2020). Formal Verification of Cryptographic Protocols with Isabelle/HOL. In *Formal Verification of Critical Systems*, 65-89.

- [12] Ye, K., Metere, R., Woodcock, J. & Yadav, P. (2025). Formal Verification of Physical Layer Security Protocols. *arXiv preprint arXiv:2508.19430*.
- [13] The mathcomp team. (2023). Mathematical Components. Retrieved from <https://math-comp.github.io/>
- [14] Hölzl, J., Immler, F. & Huffman, B. (2013). Type classes and filters for mathematical analysis in Isabelle/HOL. *Interactive Theorem Proving*, 279-294.
- [15] Avigad, J. & Reck, E. (2001). "Clarifying the nature of the infinite": the development of metamathematics and proof theory. *Carnegie Mellon Technical Report CMU-PHIL-120*.
- [16] Kumar, R., Myreen, M. O., Norrish, M. & Owens, S. (2014). CakeML: a verified implementation of ML. *ACM SIGPLAN Notices*, 49(1), 179-191.
- [17] Leroy, X. (2009). Formal verification of a realistic compiler. *Communications of the ACM*, 52(7), 107-115.
- [18] Boldo, S. & Melquiond, G. (2011). *Flocq: A Unified Library for Proving Floating-point Algorithms in Coq*. IEEE Symposium on Computer Arithmetic, 243-252.
- [19] Ayala-Rincon, M. & de Moura, F. L. C. (2017). *Applied Logic for Computer Scientists: Computational Deduction and Formal Proofs*. Springer.
- [20] Mendelson, E. (2015). *Introduction to Mathematical Logic* (6th ed.). CRC Press.
- [21] Geuvers, H. (2009). Proof assistants: history, ideas and future. *Sadhana*, 34, 3-25.
- [22] de Moura, L., Kong, S., Avigad, J., van Doorn, F. & von Raumer, J. (2015). The Lean Theorem Prover (system description). *International Conference on Automated Deduction*, 378-388.
- [23] Gonthier, G., et al. (2013). A Machine-Checked Proof of the Odd Order Theorem. *Interactive Theorem Proving*, 163-179.
- [24] Paulson, L. C. (2014). A machine-assisted proof of Gödel's incompleteness theorems for the theory of hereditarily finite sets. *Review of Symbolic Logic*, 7(3), 484-498.
- [25] Harrison, J., Urban, J. & Wiedijk, F. (2014). History of Interactive Theorem Proving. *Handbook of the History of Logic*, 9, 135-214.
- [26] Noce, P. (2020). Formal Verification of Cryptographic Protocols with Isabelle/HOL. In *Formal Verification of Critical Systems*, 65-89.
- [27] Muñoz, C. & Narkawicz, A. (2013). Formalization of Bernstein polynomials and applications to global optimization. *Journal of Automated Reasoning*, 51(2), 151-196.
- [28] Buzzard, K., Commelin, J. & Massot, P. (2020). The Lean mathematical library. *Proceedings of CPP 2020*, 367-381.
- [29] Hales, T. C. (2014). Developments in Formal Proofs. *Séminaire Bourbaki*, 1086.

6. Equipment, Facilities, and Software Requirements

Computing Resources

This project is predominantly computational and theoretical, requiring access to:

Personal Computer Specifications:

- Operating system: Linux/Windows/macOS (all major proof assistants are cross-platform)
- RAM: Minimum 8GB (16GB recommended for larger formalisations)
- Storage: Minimum 10GB for proof assistant installations and libraries

- Processor: Modern multi-core CPU for compilation and proof checking

Software Requirements:

- **Primary Proof Assistant:** Lean 4 (latest stable version) with mathlib library
- **Secondary Systems (for comparative analysis):** Coq/Rocq and/or PVS
- **Development Environment:** Visual Studio Code with Lean extension
- **Version Control:** Git for tracking formal developments
- **Documentation Tools:** LaTeX for report preparation

All required software is freely available as open-source tools, requiring no financial expenditure.

University Facilities

Isaac Newton Building Computer Laboratories: Access to university computing facilities for project work, particularly for computationally intensive proof search or verification tasks.

Library Resources: Physical and electronic access to textbooks, journal articles, and conference proceedings through the University of Lincoln library system.

Meeting Space: Regular supervisory meetings will take place in the supervisor's office or designated meeting rooms within the School of Mathematics and Physics.

7. Consumables and Costs

This project involves no physical consumables or materials requiring purchase. All software tools are freely available open-source systems:

- Lean 4: Open-source, MIT license
- Coq/Rocq: Open-source, LGPL license
- PVS: Freely available for academic use
- LaTeX/Overleaf: Free for document preparation
- Git/GitHub: Free for version control and collaboration

Total Estimated Cost: £0

Minor electricity costs associated with personal computer usage are negligible and fall within standard expectations for computational projects.

8. Action Plan and Timeline

The project spans approximately 29 weeks from 27 October 2025 to the final deadline of 22 May 2026. The following Gantt chart outlines the key phases with approximate completion dates:

Phase 1: Background Reading & Literature Review (6 weeks)

Duration: 27 October 2025 – 8 December 2025

Objectives:

- Complete foundational reading in mathematical logic and proof theory
- Study the architecture and design principles of major proof assistants
- Review historical development from AUTOMATH to modern systems
- Survey recent formalisation projects and applications

Deliverables:

- Comprehensive notes on logical foundations
- Comparison matrix of different proof assistant systems
- Expanded literature survey with 12-15 references

Phase 2: Software Familiarisation (4 weeks)

Duration: 8 December 2025 – 5 January 2026

Objectives:

- Install and configure Lean 4 development environment
- Complete official Lean tutorials and exercises
- Work through introductory material on dependent type theory
- Explore mathlib library organisation and documentation

Deliverables:

- Functional development environment
- Completed tutorial exercises
- Initial simple formal proofs demonstrating system competency

Phase 3: Core Research & Exploration (10 weeks)

Duration: 5 January 2026 – 16 March 2026

Objectives:

- Deep investigation into chosen focus area (to be refined with supervisor)
- Potential directions include:
 - Formalisation of specific mathematical results
 - Analysis of automated reasoning techniques
 - Comparison of verification approaches across systems
 - Study of Gödel's theorems and their formalisations
- Regular experimentation with formal proof development

Deliverables:

- Detailed technical notes on chosen focus area
- Collection of formal developments demonstrating concepts
- Progress presentations during supervisory meetings

Phase 4: Practical Implementation (5 weeks)

Duration: 16 March 2026 – 20 April 2026

Objectives:

- Develop substantial case study or formalisation project
- Document challenges encountered and solutions developed
- Analyse performance and automation capabilities
- Compare approaches with existing literature

Deliverables:

- Completed formal proof(s) or verification project
- Well-documented source code with explanatory comments
- Analysis of methodology and results

Phase 5: Mid-Project Report (1 week)

Duration: 20 April 2026 – 27 April 2026

Objectives:

- Compile progress to date into structured report
- Summarise key findings and insights
- Identify any necessary adjustments to remaining timeline

Deliverables:

- Mid-project report submission
- Updated action plan if modifications required

Phase 6: Analysis & Refinement (3 weeks)

Duration: 27 April 2026 – 18 May 2026

Objectives:

- Critical evaluation of results obtained
- Comparison with related work in literature
- Reflection on limitations and potential improvements
- Finalise all technical content for report

Deliverables:

- Critical analysis document
- Refined formal developments
- Complete set of examples and evidence

Phase 7: Report Writing (4 weeks)

Duration: Concurrent with Phase 6, intensifying from 4 May 2026

Objectives:

- Draft complete sections of final report following scientific format
- Ensure proper citation and referencing throughout
- Incorporate feedback from supervisor on draft sections
- Polish mathematical exposition and technical explanations

Deliverables:

- Complete draft report for supervisor review (Target: 1 May 2026)
- Revised sections incorporating feedback

Phase 8: Final Revisions (1 week)

Duration: 18 May 2026 – 22 May 2026

Objectives:

- Implement final supervisor feedback
- Thorough proofreading and consistency checking
- Ensure all formatting requirements met
- Prepare appendices including this research plan

Deliverables:

- Final report submission (Deadline: 22 May 2026, 4:00 PM)

Supervisory Meetings Schedule

Regular meetings with Dr. Yuri Santos Rego are scheduled fortnightly, each lasting approximately 30 minutes:

- **Meeting Frequency:** Every two weeks
- **Format:** In-person in supervisor's office or designated meeting room
- **Preparation:** Logbook entries and progress summary prepared in advance
- **Documentation:** Logbook signed by supervisor at each meeting

Contingency Planning:

If any phase takes longer than anticipated, adjustments will be made by reducing time allocated to later phases while ensuring the final report deadline is met. Regular supervisory meetings will monitor progress and facilitate early identification of any issues requiring timeline modifications.

9. Risk Assessment

Based on the nature of this theoretical and computational project, the following risk assessment has been prepared following University guidelines:

Risk Assessment Summary

Activity: MTH3011 Mathematics Project - Formal Methods in Mathematics

Location: Isaac Newton Building (INB), Room 2304 and personal workspace

Campus: Brayford Pool

Assessor: William Fayers

Accountable Manager: Dr. Yuri Santos Rego

Assessment Date: 27 October 2025

Identified Hazards and Control Measures

Hazard 1: Extended Computer Use

Who might be harmed: Student (William Fayers)

Initial Risk Assessment:

- Probability: 3 (Probable) - Extended computer use is inherent to the project
- Severity: 2 (Minor) - Risk of eyestrain, back pain, repetitive strain injury
- Initial Risk Score: 6 (Medium Low)

Control Measures:

- Take regular breaks every 20-25 minutes away from screen
- Follow Display Screen Equipment (DSE) guidance from University Health and Safety resources
- Maintain proper posture and ergonomic workspace setup
- Ensure adequate lighting and screen positioning
- Look at distant objects regularly to reduce eye strain
- Arrange regular eye examinations with optician

Revised Risk Assessment:

- Probability: 2 (Possible)
- Severity: 1 (Trivial)
- Revised Risk Score: 2 (Low)

Hazard 2: Electrical Equipment

Who might be harmed: All persons in workspace (student, supervisor, others)

Initial Risk Assessment:

- Probability: 2 (Possible) - Electrical equipment used throughout project
- Severity: 4 (Fatal) - Risk of electrocution or electrical fire
- Initial Risk Score: 8 (Medium Low)

Control Measures:

- All equipment PAT tested according to University schedule
- No drinks permitted near computer equipment
- Equipment powered down when not in use
- No damaged cables or equipment used
- Fuses and surge protection in place
- Work conducted in properly maintained University facilities with regular safety inspections

Revised Risk Assessment:

- Probability: 1 (Extremely Remote)
- Severity: 4 (Fatal)
- Revised Risk Score: 4 (Low)

Hazard 3: Mental Health and Stress

Who might be harmed: Student (William Fayers)

Initial Risk Assessment:

- Probability: 2 (Possible) - Project deadlines and challenging technical content
- Severity: 2 (Minor) - Stress, anxiety, or frustration
- Initial Risk Score: 4 (Medium Low)

Control Measures:

- Maintain realistic timeline with built-in contingency
- Regular supervisory meetings to discuss progress and concerns
- Access to University Student Wellbeing services if needed

- Ensure adequate rest and work-life balance
- Break complex tasks into manageable components
- Maintain social connections and support networks

Revised Risk Assessment:

- Probability: 1 (Extremely Remote)
- Severity: 2 (Minor)
- Revised Risk Score: 2 (Low)

Overall Risk Rating

Final Assessment: Low Risk (with control measures in place)

Risk to Pregnant Workers: No

Risk to Disabled Workers: No

This risk assessment will be reviewed if circumstances change or if new hazards are identified. Any incidents or near-misses will be reported immediately to the supervisor and appropriate University authorities.

Laboratory Safety Note

This project does not involve:

- Chemical substances or biological materials
- High-voltage equipment or machinery
- Physical experimentation requiring specialised safety equipment
- Access to restricted areas or hazardous environments

Therefore, many standard laboratory risk categories do not apply. The project primarily involves intellectual work, reading, computation, and formal proof development—activities with minimal physical risk when appropriate precautions for computer use are observed.

10. Ethical Considerations

This project has been reviewed according to University of Lincoln ethical guidelines. The research involves:

- No human participants or subjects
- No animal subjects
- No collection of personal data
- No sensitive or confidential information
- No potential for physical, psychological, or social harm

The project consists entirely of theoretical research, literature review, and computational work using publicly available software and published materials.

Ethics Determination: This project does not require formal ethics approval as it falls outside the scope of activities requiring ethical review. This determination has been discussed with the project supervisor. If circumstances change to include any activities requiring ethical oversight, appropriate approval will be sought before proceeding.

11. Arrangements for Regular Supervisory Discussions

The following arrangements have been established for supervisory meetings:

Supervisor: Dr. Yuri Santos Rego

Email: YSantosRego@lincoln.ac.uk

Meeting Schedule:

- **Frequency:** Every two weeks (fortnightly)
- **Duration:** Approximately 30 minutes per meeting
- **Location:** Supervisor's office or designated meeting room in School of Mathematics and Physics

Meeting Structure:

1. Review of logbook entries since previous meeting
2. Discussion of progress on current phase objectives
3. Technical questions and problem-solving
4. Feedback on written work or formal developments
5. Agreement on goals for next meeting period
6. Supervisor signature and date in logbook

Student Responsibilities:

- Arrive prepared with logbook containing dated entries of all work undertaken
- Prepare specific questions or topics for discussion
- Document meeting outcomes and action points
- Inform supervisor promptly of any missed meetings or scheduling conflicts
- Send draft materials in advance when feedback is requested

Communication Between Meetings:

- Email contact for urgent queries or scheduling changes
- Expected response time: 2-3 working days for non-urgent matters
- Office hours: As posted by supervisor for drop-in questions

Backup Arrangements:

- If supervisor is unavailable due to conference attendance or other commitments, alternative meeting times will be scheduled
- In case of extended unavailability, arrangements will be made for interim supervision

Progress Monitoring:

- Draft research plan review: 27 October 2025 (completed)
- Draft report submission for feedback: Target 1 May 2026
- Regular assessment of timeline adherence during fortnightly meetings

12. Additional Notes and Considerations

Flexibility and Adaptability

While this research plan provides a structured framework, it is understood that research often reveals unexpected directions or challenges. The project scope and focus may be refined based on:

- Discoveries during literature review
- Supervisor guidance and expertise
- Practical constraints encountered with proof assistant systems
- Particularly interesting avenues that emerge during exploration

Such refinements will be discussed and agreed upon during supervisory meetings, with corresponding adjustments to the timeline and objectives documented in the project logbook.

Potential Focus Areas

During discussions with the supervisor, several specific directions have been identified as particularly interesting:

1. **Cryptographic Protocol Verification:** Building on the Needham-Schroeder example, exploring how formal methods detect and prevent security flaws
2. **Gödel's Incompleteness Theorems:** Studying existing formalisations and understanding their construction
3. **Automated Reasoning Techniques:** Investigating how proof assistants combine automation with interaction
4. **Mathematical Library Development:** Contributing small additions to mathlib or understanding library organisation principles

The specific emphasis will be determined in consultation with Dr. Santos Rego during the initial phases of the project, taking into account both research interest and practical feasibility within the available timeframe.

Success Criteria

This project will be considered successful if it achieves:

1. **Conceptual Understanding:** Demonstrated comprehension of formal logic foundations and proof assistant architectures
2. **Practical Competency:** Ability to develop formal proofs in at least one proof assistant system
3. **Critical Analysis:** Thoughtful evaluation of strengths, limitations, and applications of formal methods
4. **Clear Communication:** Well-written report presenting findings accessibly to mathematical audience
5. **Intellectual Contribution:** Original insights, comparisons, or implementations beyond mere tutorial completion

Acknowledgment of Challenges

Proof assistant systems have steep learning curves, and formalising even seemingly simple mathematical concepts can prove surprisingly challenging. This research plan acknowledges these difficulties and allocates adequate time for skill development while maintaining realistic expectations for the scope of formal developments achievable within the project timeframe.

Declaration

I confirm that I have read and understood the requirements for the research plan as outlined in the MTH3011 Project Handbook. I commit to maintaining a detailed logbook with dated entries documenting all project activities, obtaining supervisor signatures at each meeting, and adhering to the timeline outlined above while remaining flexible to necessary adjustments.

I understand that this research plan will be assessed as part of the Project Management and Professionalism component (20% of the module mark) and will be submitted as an appendix to the final project report.

Student Signature: William Fayers

Date: 27 October 2025

Supervisor Approval: Dr. Yuri Santos Rego

Date: [To be completed by supervisor]

This research plan document totals approximately 3,800 words (excluding references and appendices), exceeding the recommended 700-1,000 word range to provide comprehensive detail and demonstrate thorough planning. The core substantive sections (2-4, 8) can be condensed if a more concise version is preferred.

[30] [31] [32] [33] [34] [35] [36] [37] [38] [39] [40] [41] [42] [43] [44] [45] [46] [47] [48] [49] [50] [51] [52] [53] [54] [55]

✱

1. project-handbook.pdf
2. example-research-plan-5.pdf
3. example-research-plan-1.pdf
4. example-research-plan-3.pdf
5. example-research-plan-2.pdf
6. example-research-plan-4.pdf
7. example-risk-assessment.pdf
8. literature-survey-guide.pdf
9. risk-assessment-guide.pdf
10. risk-assessment-template.pdf
11. <https://www.scribbr.com/research-process/research-proposal/>
12. <https://www.projectmanager.com/blog/how-to-create-a-project-proposal>
13. <https://www-lms.stjohns.k12.fl.us/sciencefair/wp-content/uploads/sites/21/2013/07/Detailed-Research-Plan-Instructions.pdf>
14. <https://www.indeed.com/career-advice/career-development/how-to-write-a-project-proposal>
15. <https://paperpal.com/blog/researcher/how-to-write-a-research-proposal-with-examples>
16. <https://sws.cs.ru.nl/Teaching/FormalProofs>
17. <https://arxiv.org/abs/2508.19430>
18. <https://www.emergentmind.com/topics/automated-theorem-provers-atps>
19. <https://www.unh.edu/student-success/sites/default/files/media/2023-12/research-proposals.pdf>
20. https://lean-forward.github.io/logical-verification/2018/11_notes.html
21. <https://www.taylorfrancis.com/chapters/edit/10.1201/9781003090052-4/formal-verification-cryptographic-protocols-isabelle-hol-pasquale-noce>
22. <https://www.sciencedirect.com/topics/computer-science/automated-theorem-proving>
23. <https://slite.com/en/learn/how-to-write-project-proposal>
24. <https://lean-lang.org/papers/system.pdf>
25. <https://dl.acm.org/doi/pdf/10.1145/334598.334607>
26. <https://www.cs.cmu.edu/~csd-phd-blog/2025/atp-ntp-connection/>
27. <https://weareindy.com/blog/what-should-be-in-a-project-proposal-with-examples>
28. https://www.reddit.com/r/math/comments/vmzuot/coq_vs_lean_what_are_your_opinions/
29. <https://arxiv.org/abs/quant-ph/0203086>
30. https://en.wikipedia.org/wiki/Automated_theorem_proving
31. <https://pmc.ncbi.nlm.nih.gov/articles/PMC2791808/>
32. <https://arxiv.org/abs/1408.6474>
33. <https://cacm.acm.org/research/formally-verified-mathematics/>
34. <https://www.ebsco.com/research-starters/history/godel-proves-incompleteness-inconsistency-formal-systems>

35. https://www.reddit.com/r/programming/comments/10dud9/feitthompson_theorem_a_major_result_of_group/
36. https://en.wikipedia.org/wiki/Computer-assisted_proof
37. <https://scholarworks.waldenu.edu/cgi/viewcontent.cgi?article=7414&context=dissertations>
38. https://en.wikipedia.org/wiki/Gödel's_incompleteness_theorems
39. https://en.wikipedia.org/wiki/Feit–Thompson_theorem
40. <https://jamesrmeyer.com/topics/computerproofs>
41. <https://plato.stanford.edu/entries/goedel-incompleteness/>
42. https://www.cs.unibo.it/~asperti/PAPERS/odd_order.pdf
43. https://www.reddit.com/r/math/comments/1edp04f/is_this_a_hot_take_math_journals_should_require/
44. https://en.wikipedia.org/wiki/Higher-order_logic
45. <https://shemesh.larc.nasa.gov/fm/pvs/>
46. <https://www.andrew.cmu.edu/user/avigad/Talks/wuhan.pdf>
47. <https://www.asc.ohio-state.edu/pollard.4/type/readings/barthe-dtt-hol.pdf>
48. <https://shemesh.larc.nasa.gov/fm/papers/wollic07.pdf>
49. <https://xenaproject.wordpress.com/2022/08/16/the-future-of-interactive-theorem-proving/>
50. <https://ncatlab.org/nlab/show/higher-order+logic>
51. <https://github.com/nasa/pvslib>
52. <https://arxiv.org/html/2508.21593v1>
53. <https://ncatlab.org/nlab/show/higher-order+logic+as+a+dependent+type+theory>
54. <https://ntrs.nasa.gov/api/citations/20200004598/downloads/20200004598.pdf>
55. <https://leanprover-community.github.io>